

Benefits of Resource Disaggregation

Michelle A. Marzoev
Cornell University

Rachit Agarwal
Cornell University

Abstract

The slowdown of Moore’s law has led to surfacing of several fundamental limitations of today’s server-centric architectures. As a result, a new computing paradigm is emerging — a disaggregated architecture, where each resource type is built as a standalone “blade” and a network fabric interconnects the resource blades. While beneficial from the computer architecture perspective, conventional wisdom suggests that decoupling of resources will lead to performance degradation for legacy systems and applications. Thus, widespread adoption of such architectures is currently gated on systems community building systems and applications optimized for this emerging computing paradigm.

We present preliminary evidence that the above blockage may be unfounded. Our main insight is that disaggregated architectures offer finer-grained resource multiplexing and improved “packing” of jobs compared to server-centric architectures. We show that, for many legacy systems and applications, performance degradation due to resource disaggregation can be completely offset by improvements due to higher resource utilization in disaggregated architectures.

1 Introduction

Server-centric architectures, where each server tightly integrates a small amount of the computing resources (CPU, memory, storage) onto a single motherboard, have been the cornerstone of cloud infrastructure. The end of Dennard’s scaling [17, 18] and the slowdown of Moore’s Law [14], however, has recently led to a paradigm shift towards a *disaggregated* architecture [1, 2, 4, 6, 7, 10, 12], where each resource type is built as a standalone disaggregated ‘blade’¹ and a network fabric interconnects the resource blades.

The qualitative benefits of resource disaggregation are rather well understood in computer architecture community [13, 27, 28]. In particular, CPU, memory, storage, I/O and networking technologies exhibit very

different trends in terms of cost, performance and power scaling [5, 9, 11]. Disaggregated architectures, by decoupling the resources, overcome some of the fundamental limitations of server-centric architectures (*e.g.*, the memory-capacity wall making CPU-memory collocation unsustainable [27]). Several recent studies suggest that this would allow disaggregated architectures to have an order of magnitude higher resource density than server-centric architectures [15, 24]. Unsurprisingly, these qualitative benefits have led major hardware manufacturers and cloud service providers to already begin developing disaggregated hardware — *e.g.*, Intel [7], Google [4], Facebook [2], Microsoft [10], HPE [6], Huawei [1] and SeaMicro [12].

While beneficial from an architectural perspective, disaggregating resources alters several assumptions that have thus far guided the design and optimization of existing systems, networks and applications (*e.g.*, CPU-storage collocation, high CPU-memory bandwidth, storage hierarchy, data locality, failure models, etc.). Conventional wisdom suggests that disaggregating resources will lead to performance degradation for legacy systems and applications. The problem however is that re-architecting systems and applications to efficiently operate on a new hardware architecture takes years. Thus, widespread adoption of and capitalizing the aforementioned benefits of disaggregated architectures is currently gated on systems community building systems and applications optimized for this emerging computing paradigm.

We present preliminary evidence that the above blockage may be unfounded. Our main insight is that, besides the qualitative benefits, disaggregated architectures also enable finer-grained statistical resource multiplexing and improved “packing” of jobs on available resources when compared to server-centric architectures with same resource capacity (§3). Based on this insight, we optimistically argue that the above conventional wisdom may not be precise — for many existing systems and applications, performance degradation due to resource disaggregation may be completely offset by improvements due to higher resource utilization in dis-

¹An exception is CPU blade, which retains certain amount of DRAM that acts as a cache for larger remote storage [19].

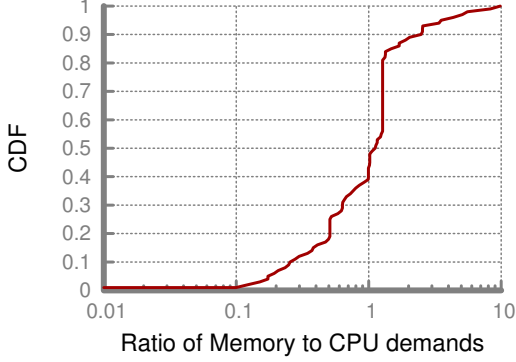


Figure 1: The memory-to-CPU requirements for tasks in Google’s cluster is highly skewed (public trace [3]).

aggregated architectures. Furthermore, higher resource density offered by disaggregated architectures may in fact lead to performance improvements. If true, this would be extremely exciting since one could achieve both qualitative and quantitative benefits of disaggregated architectures even for legacy systems and applications (as opposed to waiting for re-architecting them)!

2 Server-centric architectures

We discuss a number of factors that lead to resource underutilization and sub-optimal application performance in server-centric architectures.

2.1 Batch Processing

For batch processing jobs, the sub-optimal performance is essentially due to high underutilization of resources. We discuss a few reasons below.

Skew in resource requirements. The ratio of memory-to-CPU requirements of jobs/tasks in production clusters spread over more than two orders of magnitude (e.g., Figure 1 shows results for Google [3] workload; Facebook [20] and Microsoft [21] show similar trends). This “skew” results in high resource underutilization in server-centric architectures [29]. For instance, consider the example shown in Figure 2: the job has three tasks with skewed memory-to-CPU demands. An optimal schedule for the server-centric case is shown on the left, and has a job completion time (JCT) of 2. Moreover, irrespective of how these tasks are scheduled, server-centric architectures will result in more than 35% wastage of resources. Such resource underutilization increases the overall JCT (due to tasks spending more time waiting to be scheduled), resulting in sub-optimal application performance. We discuss the case of disaggre-

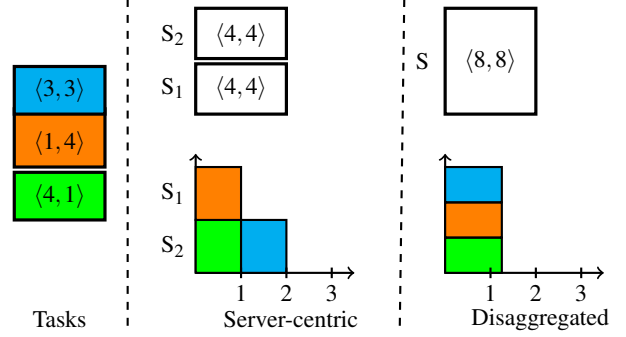


Figure 2: Resource disaggregation can lead to performance improvement of legacy systems and applications. $\langle x, y \rangle$ denotes x CPU units and y memory units. We have a job with three tasks shown on the left. S_1, S_2 are two servers. Suppose each task takes unit time on server-centric architectures and has 25% performance degradation due to resource disaggregation. The x-axis shows the time slots. The case for server-centric is discussed in §2.1; the case for disaggregated architectures is discussed in §3.1.

gated architecture in next section.

Multi-resource Packing. Besides skew, there is another fundamental reason for underutilization of resources in server-centric architectures — scheduling jobs on today’s servers requires solving a multi-dimensional resource packing problem, which is known to be a computationally hard problem [20, 21, 30]. Thus, heuristics are used in practice; it is known that even state-of-the-art heuristics can pack jobs/tasks in a manner than the resulting resource utilization is significantly worse than the optimal (see [21] and references within).

2.2 Interactive Queries

For interactive queries (e.g., key-value stores), co-location of CPU and storage in server-centric architectures avoids an additional round-trip time required in disaggregated architectures. However, as outlined in several studies [16, 26], it is more beneficial to optimize for overall system throughput — the number of queries answered per second — as long as the queries can be answered with sub-millisecond latency. Indeed, several recent studies [16, 19] show that this is an interesting tradeoff to make.

In the context of interactive query systems, resource under-utilization is because of skew in popularity of objects [25]. That is, machines storing “hot” objects receive significantly more requests than machines storing “cold” objects. Hence, the CPU at cold machines see extremely low utilization. Ideally, the requests should be load balanced across CPUs in the system.

3 Disaggregation Benefits

In this section, we make a case that disaggregated architectures offer improved resource utilization when compared to server-centric architectures, thus resulting in higher application-level performance.

3.1 Batch Processing

Disaggregating CPU from storage will most likely increase the *execution time* of individual tasks in a job when compared to server-centric architectures. However, the overall completion time of a job depends not only on the execution time, but also on the *waiting time* — time spent by the job while waiting to be scheduled. Thus, for cases when the increase in execution time can be offset by reduction in waiting time, legacy systems and applications will not suffer from degraded overall performance on disaggregated architectures. Our study builds upon two main insights.

Improved statistical resource multiplexing. Our first insight is that decoupling of resources in disaggregated architectures allows more fine-grained statistical resource multiplexing than server-centric architectures. As a result, the available resources can be utilized more efficiently leading to improvement in job performance.

For instance, consider again the example in Figure 2. Recall that for server-centric architectures, the optimal JCT in this example is 2. Now suppose that due to resource disaggregation, each task observes 25% degradation in execution time (compared to server-centric architectures); thus, each task now has an execution time of 1.25 units. However, also note that resource disaggregation now enables two larger “resource pools” — one with 8 CPU units and the other with 8 memory units. Thus each of these tasks can be run in parallel on the disaggregated hardware (as shown on the right side of the figure). Each task does observe 25% degradation in execution time but due to better resource utilization, the job finishes at time 1.25, an overall *improvement* of $1.6\times$ in job completion time when compared to server-centric architectures.

Using traces from Google production cluster, we show in §4 that for most tasks, improved resource packing in disaggregated architectures leads to improvement in overall JCT (both at median and high percentiles) when compared to server-centric architectures. Note in the above example that as long as tasks have less than 100% performance degradation due to disaggregation (which is shown to be the case in [19]), the job will have better performance on disaggregated architectures compared to server-centric architectures.

Simplified resource packing. Our second insight is that decoupling of resources in disaggregated architectures leads to large pools of individual resources, and this in itself simplifies the multi-resource packing problem discussed earlier. Specifically, the problem of resource packing in disaggregated architectures reduces to a special case of packing problem where each job/task takes an extremely small fraction of the entire resource. This is similar to bin packing with extremely small items [8], and a simple greedy algorithm provably computes a packing arbitrarily close to optimal. We discuss the greedy algorithm below, and evaluate the performance of the algorithm in §4.

Our algorithm for multi-resource packing on disaggregated architectures is extremely simple: it iterates through all the tasks in the order of their arrival and greedily packs as many tasks as possible. We now give an intuitive description of why this simple greedy algorithm achieves resource utilization arbitrarily close to optimal. Suppose the resource demand across all tasks and is a $\epsilon \ll 1$ fraction of the resource capacity (across all resources); that is, for each task, the CPU and the memory demand is ϵ fraction of the capacity of the CPU and memory pools, respectively. To see why this algorithm performs close to optimal, consider the first task that cannot be packed — it must be the case that all the resources in the disaggregated resource pools have remaining capacity less than ϵ since otherwise the task would have been packed. Thus, the resource utilization is $(1 - \epsilon)$. Note that the above algorithm works for any arbitrary ordering of tasks, and thus can operate in an online manner (as new tasks arrive) while providing the same packing efficiency guarantees.

Of course, one can use the above greedy algorithm for server-centric architectures as well. The reason this algorithm works well for disaggregated architectures is because when the resources from individual servers are combined into resource pools, the value of ϵ becomes much smaller when compared to that in server-centric architectures. For instance, consider a rack with 40 servers, each with unit amount of CPU and memory. Then, if a task requires 0.5 units of CPU and memory, the value of ϵ is 0.5 leading to a bound of at least 50% resource utilization. On the other hand, if resources from all servers are combined into a resource pool, the value of ϵ reduces by a factor of 40 to 0.0125, leading to an improved bound of 98.75% resource utilization.

We note that higher software overheads will lead to an improved scenario for legacy systems over disaggregated architectures since the applications will observe lower degradation in execution time [19].

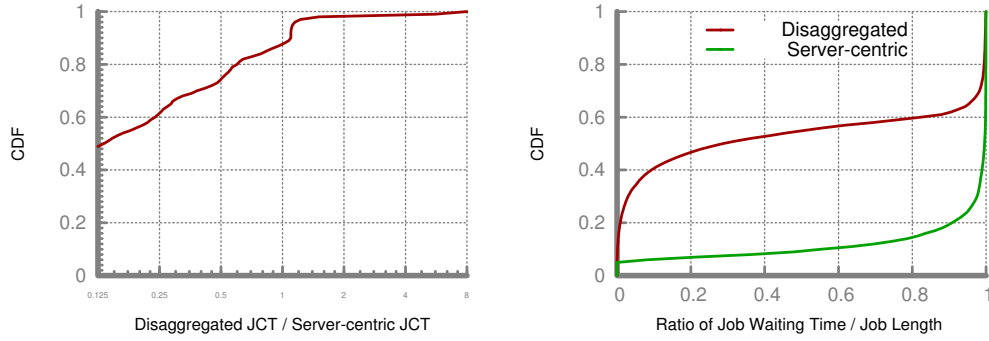


Figure 3: Comparison of server-centric and disaggregated architectures for batch processing jobs. The left figure shows that more than 85% of the jobs are sped up in disaggregated architectures with almost 60% of the jobs being sped up by $4\times$ and almost 50% of the jobs sped up by $8\times$. The right figure outlines the reason for the speed up — specifically, jobs see significantly lower waiting time (due to improved resource packing leading to faster scheduling) in disaggregated architectures. See discussion in §4.

3.2 Interactive Queries

For interactive queries, disaggregated architectures offer the same benefits as so called serverless architectures [23]. Specifically, the client requests can be directed to any of the CPU blades, which then forward the request to the appropriate storage node and return the client query. Indeed, this requires an additional round-trip time between CPU nodes and storage nodes; while this additional round-trip time can be insignificant compared to client-server time, this leads to significant well balanced load across CPUs (as we show in next section).

4 Preliminary Evaluation

We now compare the performance of legacy applications over disaggregated architectures against server-centric architectures. We use batch processing applications (e.g., MapReduce and Spark) for our evaluation due to availability of real-world traces [3]; we discuss the case for other applications (e.g., interactive queries and streaming systems) in §5. We start by describing our evaluation setup, and then discuss evaluation results.

Evaluation Setup. We use simulations for our evaluation. There are four inputs to the simulator. The first input is a set of jobs, comprising of multiple tasks each having resource (CPU, memory and storage) demands and execution time for server-centric architectures taken from a 29 day MapReduce trace from Google cluster [3]. The second input is a set of servers comprising of CPU, memory and storage resources; we consider a rack comprising of 40 servers with configuration from the Google trace. The third input is the set of resources in disaggregated cluster; for this input, we simply take the total

number of CPUs, total amount of memory and storage comprising the 40 servers in server-centric architecture and use that as the available resource pools in disaggregated architectures (that is, we ignore the qualitative benefit of higher resource density in disaggregated architectures which will further improve the performance numbers). Finally, the fourth input is a *slowdown* factor that each individual task observes due to resource disaggregation; we use a slowdown factor of 10%, a significant overestimate compared to those observed for batch processing jobs in previous studies [19].

We compare three schemes in terms of job completion times and resource utilization: (1) Tetris, the state-of-the-art multi-resource packing algorithm [22] for server-centric architectures; (2) Tetris over disaggregated architectures; and (3) our algorithm from §3.1 over disaggregated architectures.

Discussion of results (Figure 3). The left plot shows the CDF of the ratio of job completion time for disaggregated (using greedy packing) and for server-centric architectures (using Tetris packing). We observe that while some jobs take longer to finish in disaggregated architectures, around 70% of the jobs finish at least $2\times$ faster and around 50% of the jobs finish $8\times$ faster in disaggregated architectures. A very large fraction of the jobs observe significant reduction. The overall job completion time reduces by $7.43\times$ at median and by $4.16\times$ on an average. This is significant given that we do not even consider the qualitative benefit of higher resource density in disaggregated architectures, which will lead to further improvement in performance.

The above results can be attributed to higher resource utilization in disaggregated architectures as shown in

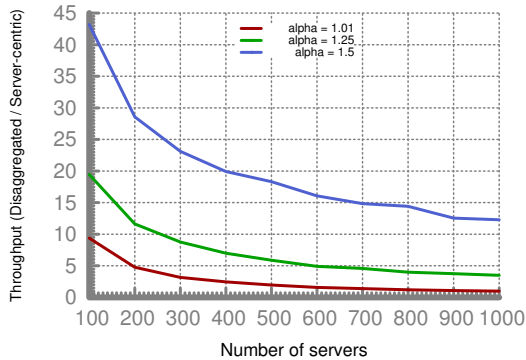


Figure 4: For skewed workloads, disaggregated architectures lead to 2 – 10 $\times$ benefits in overall system throughput (number of queries answered) when compared to server-centric architectures for interactive query systems. The throughput benefits improve as more objects reside on fewer servers due to improved load balancing across CPUs.

the right plot in Figure 3. Indeed, the greedy packing over disaggregated resources almost always has higher resource utilization compared to Tetris packing over server-centric architectures; this leads to significantly reduced “waiting time” for the jobs (around 4 $\times$) since jobs can be scheduled earlier than the time in server-centric architectures.

Overall, our evaluation shows that for many legacy systems and applications, performance degradation due to resource disaggregation may be completely offset by improvements due to higher resource utilization in disaggregated architectures. In fact, in many cases, legacy systems and application may observe improved performance over disaggregated architectures.

Interactive Queries. Figure 4 shows that disaggregated architectures offer as much as an order of magnitude higher throughput than server-centric architectures for interactive query systems (at the cost of one extra round-trip time, as discussed earlier).

To perform this experiment, we use a key-value store with 1Billion key-value pairs distributed across a varying number of machines (x-axis in the plot). We then generate requests over these key-value pairs using a zipf distribution with varying skew rate (smaller is more skewed).

For smaller number of machines, more key-value pairs are on “hot” servers thus increasing the load on the server. This, in return, makes the hot server extremely over-loaded and the cold server being under-utilized. For disaggregated context, this load is well distributed across the various CPU resources and hence, we see much higher throughput. As the number of servers increase, the key-value pairs are distributed across more

servers and disaggregation leads to lower benefits (as expected). Note that in practice, increase in number of servers comes with increase in number of objects; in this scenario, disaggregated architectures will continue to offer benefits as with smaller number of servers in Figure 4.

5 Research Agenda

Capitalizing on *all* the benefits of disaggregated architectures will require resolving several challenges at the intersection of systems, networks and algorithms. Our study shows that some of these benefits can be exploited today — cloud providers can begin efforts towards adoption of disaggregated hardware; as optimized systems and applications evolve, higher performance improvements can be extracted from this new emerging paradigm. However, much remains to be done. We now outline a few potential research directions.

5.1 Broader classes of applications

Our study primarily focuses on batch processing jobs like MapReduce and Spark, and interactive query systems like key-value stores. It would be interesting to incorporate broader classes of applications into our study, including more systems on interactive queries (*e.g.*, databases), streaming systems and graph stores. A priori, it is conceivable that we may observe results similar to those in Figure 3 — resource disaggregation leads to similar degradation in execution time for state-of-the-art systems for these applications [19]. There are two challenges, though. First, using current networking technologies will lead to increased latency for interactive queries when the entire dataset fits in memory for server-centric architectures (as discussed earlier). This may be tolerable for some applications (*e.g.*, as long as overall latency is less than what humans can perceive); for other applications, our study may lead to new insights about the desired performance from the network fabric for disaggregated architectures. Second, higher resource utilization in disaggregated architectures may allow higher throughput (number of queries answered per second) and may be interesting for applications where trading off a little latency for higher throughput may be desirable. Our current focus is on extending our study in this direction.

5.2 Higher Resource Density

Our study did not focus on the main benefit offered by resource disaggregation — higher resource den-

sity [15, 24]. A priori, it is conceivable that higher resource density will translate into even improved job completion time for each job (via even more reduction in waiting time). Characterizing the achievable benefits for higher density disaggregated hardware using real-world datasets remains an intriguing direction.

5.3 Design Space Exploration

The overall performance of legacy systems and applications on disaggregated architectures depends on performance degradation of individual tasks due to disaggregation as well as performance improvement due to higher resource utilization. The former depends on the underlying disaggregated architecture design. Existing disaggregated architecture prototypes vary significantly in underlying design decisions. For instance, across prototypes, the on-board DRAM to CPU ratio varies from 15% to 85% of server-centric architectures [1, 6, 7]. The storage hierarchy in these prototypes also varies from a three-level (comprising of on-chip cache, on-board DRAM, and large remote banks of DRAM) [7, 15] to six-level (adding large remote banks of NVRAM, SSD and disk) [1, 6]. Finally, the interconnections vary from silicon photonics [13] to PCIe-based fabric [1] to optical and electrical links and switches [7, 19]. These significantly different designs will likely have significantly different impact on the results of Figure 3. In addition to generalizing our study to incorporate various design parameters, we would also like to explore several interesting performance-related research questions:

- **What is the ideal scale of disaggregation?** Here, the two options are *cloud-scale* or *rack-scale* disaggregation. In the former, applications can use resources from any blade across the entire cloud, while the latter restricts the applications to blades within a rack. This can be done either at the systems level using a smart scheduler or at the hardware level by confining the scope of resource pooling. Intuitively, the tradeoff is that cloud-scale disaggregation potentially has higher performance degradation (due to further separation between compute and storage) but higher resource utilization than rack-scale disaggregation. However, exploring this tradeoff and exploring the impact on application performance remains open.
- **How should the resource blades and the network fabric be organized within and across racks for optimal application performance?** Another design aspect related to scale of disaggregation is placement of resource blades and organization of network fabric. Is it better to distribute the resource blades across racks or shall racks contain resources of one type

only? Shall we design network fabrics that are homogeneous (that is, same fabric within and across racks) or heterogeneous (*e.g.*, it may be desirable that the fabric within a rack provides higher bandwidth and lower latency than the fabric across racks)?

5.4 Re-architecting Applications

Wide-spread adoption of disaggregated architectures seems inevitable, as demonstrated by tremendous recent efforts from industry [1, 2, 4, 6, 7, 10, 12]. Our point is not that optimizing systems for these architectures is not a direction worth pursuing; in fact, we believe that the time is ripe for systems community to build the infrastructure and to re-architect systems, networks and applications for this emerging computing paradigm. However, the two *need not be coupled* — cloud providers can begin efforts towards adoption of disaggregated hardware, and can incorporate optimized systems and applications as and when they evolve.

References

- [1] Dc 3.0 architecture. <http://pr.huawei.com/en/news/hw-423134-3.0.htm#.V1oPxkKJndk>.
- [2] Facebook disaggregated rack. <http://goo.gl/6h2Ut>.
- [3] Google cluster data, version 2. <https://github.com/google/cluster-data>.
- [4] Google patent: Disaggregation of server components in a data center. <http://www.google.com/patents/US20140164669>.
- [5] Graphics processing unit. https://en.wikipedia.org/wiki/Graphics_processing_unit.
- [6] HP The Machine. <http://www.hpl.hp.com/research/systems-research/themachine/>.
- [7] Intel RSA. <http://www.intel.com/content/www/us/en/architecture-and-technology/rsa-demo-x264.html>.
- [8] Lecture notes on bin packing. https://www2.informatik.hu-berlin.de/alcox/lehre/lvws1011/coalg/bin_packing.pdf.
- [9] "memristor". <https://en.wikipedia.org/wiki/Memristor>.

- [10] Microsoft rack-scale computing. <http://research.microsoft.com/en-us/projects/rackscale/>.
- [11] Non-volatile random-access memory. https://en.wikipedia.org/wiki/Non-volatile_random-access_memory.
- [12] Seamicro technology overview. http://seamicro.com/sites/default/files/SM_T001_64_v2.5.pdf.
- [13] K. Asanović. Firebox: A hardware building block for 2020 warehouse-scale computers. FAST '14.
- [14] R. Colwell. The Chip Design Game at the End of Moore's Law. In *HotChips-25*, 2013.
- [15] P. Costa. Towards rack-scale computing: Challenges and opportunities. <https://tinyurl.com/zza4yee>.
- [16] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang, et al. The snowflake elastic data warehouse. In *Proceedings of the 2016 International Conference on Management of Data*, pages 215–226. ACM, 2016.
- [17] H. Esmailzadeh, E. Blem, R. St. Amant, K. Sankaralingam, and D. Burger. Power challenges may end the multicore era. In *Communications of the ACM, Vol. 56 No. 2, Pages 93-102*.
- [18] H. Esmailzadeh, E. Blem, R. St. Amant, K. Sankaralingam, and D. Burger. Dark silicon and the end of multicore scaling. In *Proc. of the 38th Annual International Symposium on Computer Architecture, ISCA '11*, pages 365–376, New York, NY, USA, 2011. ACM.
- [19] P. Gao, A. Narayan, S. Karandikar, J. Carreira, S. Han, R. Agarwal, S. Ratnasamy, and S. Shenker. Network support for disaggregated datacenters. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016.
- [20] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2011.
- [21] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella. Multi-resource packing for cluster schedulers. In *ACM SIGCOMM Computer Communication Review*, 2014.
- [22] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella. Multi-resource packing for cluster schedulers. In *ACM SIGCOMM Computer Communication Review*, volume 44, pages 455–466. ACM, 2014.
- [23] S. Hendrickson, S. Sturdevant, T. Harter, V. Venkataramani, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. Serverless computation with openlambda. *Elastic*, 60:80, 2016.
- [24] K. Keeton. The machine: An architecture for memory-centric computing. <http://www.mcs.anl.gov/events/workshops/ross/2015/slides/ross2015-keeton.pdf>.
- [25] A. Khandelwal, R. Agarwal, and I. Stoica. Blowfish: Dynamic storage-performance tradeoff in data stores. In *USENIX Symposium on Network System Design and Implementation (NSDI)*, 2016.
- [26] H. Lim, D. Han, D. G. Andersen, and M. Kaminsky. Mica: A holistic approach to fast in-memory key-value storage. *management*, 15(32):36, 2014.
- [27] K. Lim, J. Chang, T. Mudge, P. Ranganathan, S. K. Reinhardt, and T. F. Wenisch. Disaggregated memory for expansion and sharing in blade servers. *ISCA '09*.
- [28] S. Novakovic, A. Daglis, E. Bugnion, B. Falsafi, and B. Grot. Scale-out NUMA. In *Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014.
- [29] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing*, page 7. ACM, 2012.
- [30] G. J. Woeginger. There is no asymptotic ptas for two-dimensional vector packing. *Information Processing Letters*, 64(6):293–297, 1997.